

GymControl - Guía Paso a Paso de Modificaciones

Plan de Trabajo Ajustado a la Normativa del TP Integrador (Programación 1 - UADE)

Esta guía detalla el orden exacto de implementación para adaptar el código existente de **GymControl** a las consignas obligatorias del Trabajo Práctico, priorizando la refactorización a matrices y dejando el tema de **Expresiones Regulares (Regex)** para el final (post-clase).

Paso 1: Estructurar las Matrices Principales (Datos Base)

Reemplazar las múltiples listas paralelas unidimensionales por 3 matrices principales (listas de listas):

```
# 1. Matriz de Socios: [codigo, nombre, edad, tipo]
matriz_socios = [
    [101, 'Juan Perez', 28, 1],
    [102, 'Maria Juana', 32, 3]
]

# 2. Matriz de Actividades: [codigo, nombre, nivel, cupo_disponible]
matriz_actividades = [
    [201, 'Yoga', 1, 15],
    [202, 'Crossfit', 3, 10]
]

# 3. Matriz de Inscripciones: [codigo_insc, codigo_socio, codigo_clase, asistencias, estado]
matriz_inscripciones = [
    [301, 101, 205, 8, 1],
    [302, 102, 204, 3, 1]
]
```

Paso 2: Adaptar Funciones ABM (Socios, Clases e Inscripciones)

- [] **Módulo Socios:** Modificar `sumar_afiliados()` para hacer `matriz_socios.append([codigo, nombre, edad, tipo])`. Actualizar `eliminar_afiliados()`, `modify_affiliate()` y `list_affiliates()` para consultar y modificar por índices (`fila[0]`, `fila[1]`, etc.).
- [] **Módulo Clases:** Modificar `sumar_clase()`, `eliminar_clase()`, `modify_clase()` y `list_clases()` para operar sobre filas de `matriz_actividades`.
- [] **Módulo Inscripciones:** Modificar `alta_inscripcion()`, `baja_inscripcion()`, `modify_inscripcion()` y `list_inscripciones()` para trabajar sobre filas de `matriz_inscripciones`.

Paso 3: Adaptar Algoritmos de Búsqueda y Ordenamiento

- [] **Búsquedas:** Adaptar `buscar_socio_secuencial()` para buscar el código en la primera columna (`fila[0]`) de `matriz_socios`. Adaptar `buscar_clase_binaria()` para buscar sobre filas de `matriz_actividades`.
- [] **Ordenamientos:** Adaptar `ordenar_socios_por_edad()`, `ordenar_clases_por_nivel()` y `ordenar_inscripciones_por_asistencias()` para que intercambien **filas completas** en lugar de modificar listas paralelas.

Paso 4: Agregar las 2 Funciones Nuevas Requeridas

1. Función para cumplir con 'reduce' (Reporte Estadístico Simple):

```
from functools import reduce

def calcular_total_asistencias():
    total = reduce(lambda acum, fila: acum + fila[3], matriz_inscripciones, 0)
    print(f'Total general de asistencias registradas: {total}')
```

2. Función para cumplir con el listado por actividad de la propuesta:

```
def listar_socios_de_actividad():
    codigo_clase = pedir_entero('Ingrese el código de la clase: ')
```

```
inscripciones = list(filter(lambda fila: fila[2] == codigo_clase and fila[4] == 1,
matriz_inscripciones))
print(f'Socios inscriptos en la clase {codigo_clase}:')
for insc in inscripciones:
    pos_socio = search_affiliate_position(insc[1])
    print(f'- {matriz_socios[pos_socio][1]} (Código: {insc[1]})')
```

Paso 5: Expresiones Regulares (re) — [Post-clase de mañana]

Una vez vista la clase teórica de expresiones regulares, importar re e incorporar la función de validación:

```
import re

def es_numero_valido(cadena):
    return bool(re.match(r'^\d+$', str(cadena).strip()))

# Usar es_numero_valido() en pedir_entero() y en las validaciones de entrada.
```